

PINNs Tutorial

Ahmad Azzeh

May 2026

Contents

1	Multi-Layer Perceptrons	3
1.1	The Perceptron — One Neuron, One Decision	3
1.1.1	What a Perceptron Does	3
1.1.2	Why Perceptrons Are Needed — The Nonlinearity Problem	3
1.1.3	How a Perceptron Weighs Competing Outcomes	4
1.2	Architecture	6
1.3	The Forward Pass	6
1.4	Activation Functions	7
1.5	Why Tanh and Not ReLU for PINNs?	8
1.6	Loss Functions and Training Objective	8
1.7	Backpropagation and Gradient Descent	9
1.8	Fourier Feature Embeddings	10
1.8.1	Why Does a Standard MLP Struggle with Sharp Gradients?	10
1.8.2	What Are Fourier Feature Embeddings?	10
1.8.3	So the Network Only Learns the Amplitudes — Not the Frequencies?	10
2	Backpropagation and Automatic Differentiation	11
2.1	The Computation Graph	12
2.2	Backpropagation — Differentiating with Respect to Weights	12
2.2.1	The Chain Rule	12

2.2.2	Worked Example: Two Weights, One Step	13
2.3	Complex Worked Examples	14
2.3.1	Example 1 — $f(x, y, z) = (x + y)z$	14
2.3.2	Example 2 — Sigmoid Neuron	15
2.3.3	Example 3 — Autodiff Exactness: $f(x) = \sin(x) e^x$	17
2.3.4	Summary: What the Examples Demonstrate	18
2.4	Automatic Differentiation — Differentiating with Respect to Inputs	19
2.4.1	Three Ways to Differentiate	19
2.4.2	What Makes Autodiff Different from Backpropagation	20
2.4.3	Computing PDE Derivatives in PyTorch	20
2.4.4	Why Autodiff Is Exact	22
2.5	How Backpropagation and Autodiff Work Together in PINN Training	22
3	Convolutional Neural Networks	23
3.1	What Is a CNN?	23
3.2	Why Does This Thesis Use an MLP and Not a CNN?	24
3.3	Summary: MLP vs CNN for Continuous Coordinate Input	25
4	Automatic Differentiation	26
4.1	Three Ways to Differentiate — Why Only One Works for PINNs	26
4.2	How Autodiff Computes PDE Residuals in PyTorch	27

1 Multi-Layer Perceptrons

A **Multi-Layer Perceptron** (MLP) is the foundational architecture of modern neural networks and the building block on which Physics-Informed Neural Networks are constructed. Understanding it in detail is essential before any PINN concept can be introduced.

1.1 The Perceptron — One Neuron, One Decision

The **perceptron** is the fundamental building block of every neural network. It is a single artificial neuron that takes a set of numerical inputs, weighs each one according to its importance, sums the result, and passes it through an *activation function* to produce an output. Before understanding how layers of neurons work together, it is necessary to understand what one neuron does on its own.

1.1.1 What a Perceptron Does

A perceptron with n inputs $x_1, x_2, \dots, x_n$ computes:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \sum_{i=1}^n w_ix_i + b \quad (1)$$

$$a = \sigma(z) \quad (2)$$

Each weight w_i controls how much input x_i influences the result. A large $|w_i|$ means the perceptron pays close attention to x_i ; a value near zero means it effectively ignores it. The bias b shifts the decision threshold — without it, the perceptron can only represent functions that pass through the origin. The activation function σ introduces nonlinearity, discussed further in Section 1.1.2.

1.1.2 Why Perceptrons Are Needed — The Nonlinearity Problem

For simple, linearly separable problems, a single perceptron (or even a straight line drawn through a scatter plot) is sufficient. Imagine a dataset of points on a graph: if all points of class A sit on one side and all points of class B on the other, a single

straight line separates them perfectly. The perceptron learns the equation of that line through its weights w_i and bias b .

However, many real-world problems are not linearly separable. Consider a dataset where class A points cluster in two opposite corners of the graph, and class B points cluster in the other two — no single straight line can separate them. This is the classical *XOR problem*, and it is the reason a single perceptron is insufficient for most practical tasks.

The solution is to stack multiple perceptrons into layers. Each additional layer allows the network to draw one more *curved* or *composite* decision boundary. With enough neurons and layers, the network can separate arbitrarily complex data distributions — the universal approximation theorem formalises this result [?]. The nonlinearity comes from two sources working together:

1. **Multiple neurons in parallel:** each neuron in a hidden layer draws its own hyperplane through the input space. Together they partition the space into regions.
2. **Nonlinear activation functions:** without σ , stacking layers of perceptrons is mathematically equivalent to a single linear transformation, no matter how many layers are used. The composition of linear maps is linear. The activation function breaks this collapse, giving each layer the ability to represent genuinely nonlinear relationships.

This is why a single perceptron cannot replace an MLP for the present problem. The temperature field $T(x^*, r^*, t^*)$ of a NiTi wire depends on all three coordinates in a coupled, nonlinear way — the Fourier heat equation with convective boundary conditions has no closed-form linear solution. The network must learn a nonlinear mapping from coordinates to temperature, which requires multiple perceptrons composed through nonlinear activations.

1.1.3 How a Perceptron Weighs Competing Outcomes

A perceptron does not output a binary yes/no decision — it outputs a continuous score (the pre-activation z , or after activation, a value $a \in (-1, 1)$ for $\tanh$). When the network is used for classification, these scores are converted into probabilities

by the *softmax* function, which normalises the raw scores so they sum to one.

To make this concrete, consider a perceptron that produces two competing output scores: $z_x = 2.0$ for outcome x and $z_y = 2.2$ for outcome y . The softmax probabilities are:

$$P(x) = \frac{e^{z_x}}{e^{z_x} + e^{z_y}} = \frac{e^{2.0}}{e^{2.0} + e^{2.2}} = \frac{7.389}{7.389 + 9.025} \approx 0.45 \quad (3)$$

$$P(y) = \frac{e^{z_y}}{e^{z_x} + e^{z_y}} = \frac{e^{2.2}}{e^{2.0} + e^{2.2}} = \frac{9.025}{7.389 + 9.025} \approx 0.55 \quad (4)$$

The network predicts outcome y (the higher-scoring class), but the difference is small: 55% versus 45%. This illustrates a critical property of weighted scoring — the *losing* outcome does not disappear from consideration. Its score of 2.0 still contributes a probability of 0.45, meaning the network is genuinely uncertain between the two outcomes. A score difference of only 0.2 maps to a probability difference of only 10 percentage points.

This has a direct consequence for how the network learns. During backpropagation, the gradient of the loss with respect to the weights is proportional to the prediction error. When the network is uncertain (probabilities close to 0.5 each), the gradient is large and the weights are updated substantially. When the network is confident (one probability near 1.0), the gradient is small and the weights barely change. The weights therefore encode not just *which* outcome is predicted, but *how certain* the network is — and uncertainty drives learning.

For regression tasks such as the thermal field reconstruction in this thesis, there is no softmax and no probability interpretation. Instead, the output a is directly the predicted temperature T^* , and the loss is the squared error between a and the true value. The weighting principle remains: a neuron with a large weight on input x_i is saying that x_i strongly determines the output, and a small weight is saying it barely matters.

1.2 Architecture

An MLP consists of an ordered sequence of *layers*, each containing a set of computational units called *neurons* or *nodes*. Three layer types are distinguished:

- **Input layer:** receives the raw input features. Each neuron corresponds to one input variable.
- **Hidden layers:** one or more intermediate layers that learn internal representations of the input. All neurons in one layer are connected to all neurons in the next — this is called *full connectivity* or a *dense layer*.
- **Output layer:** produces the final prediction. For scalar field reconstruction the output layer has a single neuron.

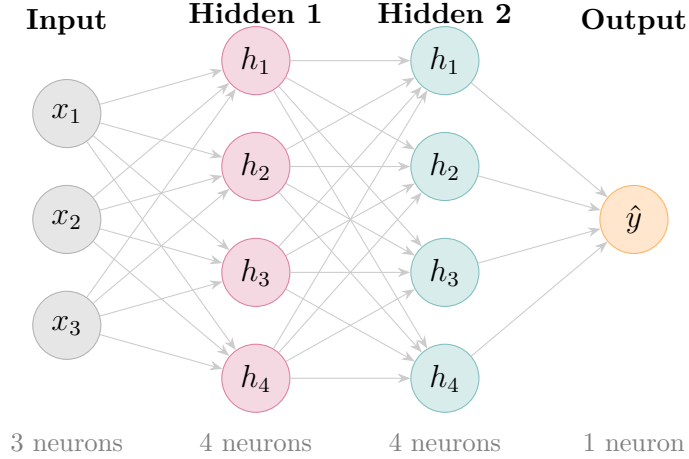


Figure 1: Architecture of a Multi-Layer Perceptron with one input layer (3 neurons), two hidden layers (4 neurons each), and one output layer. Every neuron in a layer connects to every neuron in the next layer.

1.3 The Forward Pass

Consider a single neuron j in a hidden layer receiving inputs from the previous layer. It computes a *pre-activation* value z_j as a weighted sum of its inputs plus a scalar bias b_j :

$$z_j = \sum_i w_{ij} a_i + b_j \quad (5)$$

where w_{ij} is the weight connecting neuron i in the previous layer to neuron j , and a_i is the *activation* (output) of neuron i . The neuron then applies a nonlinear *activation function* $\sigma(\cdot)$ to produce its output:

$$a_j = \sigma(z_j) \quad (6)$$

This process — computing weighted sums, adding biases, and applying the activation function — propagates from the input layer through every hidden layer to the output. This sequential computation is called the *forward pass*.

In matrix notation for a network with L layers, the forward pass is:

$$\mathbf{a}^{(l)} = \sigma\left(W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\right), \quad l = 1, \dots, L \quad (7)$$

where $W^{(l)}$ is the weight matrix of layer l , $\mathbf{b}^{(l)}$ its bias vector, and σ is applied element-wise.

1.4 Activation Functions

The nonlinearity introduced by $\sigma(\cdot)$ is what gives neural networks their expressive power. Without it, stacking layers would be equivalent to a single linear transformation — the composition of any number of linear maps remains linear. Three activation functions are relevant here:

- **ReLU** (Rectified Linear Unit): $\sigma(z) = \max(0, z)$. Computationally cheap and widely used in classification tasks, but its non-differentiability at $z = 0$ is problematic for PINNs, which require smooth second-order derivatives of the network output.
- **Sigmoid**: $\sigma(z) = 1/(1 + e^{-z})$. Smooth and bounded in $(0, 1)$, but suffers from *vanishing gradients* in deep networks.

- **Tanh:** $\sigma(z) = \tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$. Smooth, bounded in $(-1, 1)$, zero-centred, and infinitely differentiable. This is the standard activation function for PINNs ?.

1.5 Why Tanh and Not ReLU for PINNs?

The PINN loss function includes PDE residuals that require second-order spatial derivatives of the network output — specifically $\partial^2 f / \partial x^2$ and $\partial^2 f / \partial r^2$ — computed via automatic differentiation applied twice with respect to the input coordinates. For this to return a meaningful, non-zero value everywhere in the domain, the activation function must be at least C^2 (twice continuously differentiable).

ReLU is piecewise linear: its first derivative is a step function and its second derivative is zero almost everywhere, making it useless for computing PDE residuals. Tanh is C^∞ and its derivatives are well-defined and non-trivial at every point:

$$\sigma(z) = \tanh(z) \tag{8}$$

$$\sigma'(z) = 1 - \tanh^2(z) \tag{9}$$

$$\sigma''(z) = -2 \tanh(z) (1 - \tanh^2(z)) \tag{10}$$

Equation (10) is non-zero at every $z \neq 0$, ensuring that the second derivative of the network output — and therefore the PDE residual — carries meaningful curvature information throughout training.

1.6 Loss Functions and Training Objective

The MLP contains a large number of trainable parameters — all weights w_{ij} and biases b_j , collectively denoted θ . Training means finding the values of θ that minimise a *loss function* $\mathcal{L}(\theta)$ quantifying how poorly the network's predictions match the desired output.

For standard supervised regression on a dataset $\{(\mathbf{x}_k, y_k)\}_{k=1}^N$, the canonical loss is the *Mean Squared Error* (MSE):

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N} \sum_{k=1}^N \left(f(\mathbf{x}_k; \theta) - y_k \right)^2 \quad (11)$$

This is a purely data-driven objective. The PINN framework augments it with additional physics-based residual terms, discussed in Section 4.2.

1.7 Backpropagation and Gradient Descent

Minimising $\mathcal{L}(\theta)$ requires computing the gradient $\nabla_{\theta} \mathcal{L}$ — how the loss changes with respect to every parameter. This is accomplished by *backpropagation*: an efficient application of the chain rule of calculus that propagates error signals backward from the output layer to the input.

For a single weight $w_{ij}^{(l)}$ in layer l , the chain rule gives:

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} \cdot a_i^{(l-1)} \quad (12)$$

where $\delta_j^{(l)}$ is the *error signal* at neuron j in layer l , computed recursively from the layer above. This recursion is what makes backpropagation efficient: each gradient is computed once and reused.

With gradients in hand, parameters are updated by *gradient descent*:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L} \quad (13)$$

where $\eta > 0$ is the *learning rate*. Choosing η too large causes the optimiser to overshoot minima and diverge; too small and convergence is prohibitively slow. In practice, adaptive optimisers such as **Adam** ? maintain per-parameter learning rates and first/second moment estimates, providing more robust convergence than vanilla gradient descent.

1.8 Fourier Feature Embeddings

1.8.1 Why Does a Standard MLP Struggle with Sharp Gradients?

Standard MLPs exhibit a phenomenon known as *spectral bias*: gradient descent preferentially fits the low-frequency components of the solution first and converges very slowly on high-frequency spatial variations ?. Intuitively, the network’s random initialisation produces nearly-linear functions, and the optimiser finds smooth solutions because they have smaller gradient norms. High frequency features are only learned much later in training, if at all.

For the present problem, the sharp thermal gradient near the wire ends — decaying over a characteristic length $L_b \approx \sqrt{kR/2h} \approx 9.5 \text{ mm}$ — is precisely the feature that determines end-cooling accuracy and must be captured faithfully. A plain MLP will fit the bulk temperature field accurately but smear out that end gradient.

1.8.2 What Are Fourier Feature Embeddings?

The remedy is to transform the raw input coordinates *before* feeding them to the network ?. Define the Fourier feature map:

$$\boldsymbol{\gamma}(\mathbf{x}^*) = \begin{bmatrix} \sin(2\pi \mathbf{B}_1 \cdot \mathbf{x}^*) \\ \cos(2\pi \mathbf{B}_1 \cdot \mathbf{x}^*) \\ \vdots \\ \sin(2\pi \mathbf{B}_m \cdot \mathbf{x}^*) \\ \cos(2\pi \mathbf{B}_m \cdot \mathbf{x}^*) \end{bmatrix} \quad (14)$$

where $\mathbf{x}^* = (x^*, r^*, t^*)$ are the nondimensionalised input coordinates and $\mathbf{B} \in \mathbb{R}^{m \times 3}$ is a fixed random frequency matrix, sampled once from $\mathcal{N}(0, \sigma^2)$ before training and held frozen throughout. The mapping $\boldsymbol{\gamma}$ expands the three input coordinates into a $2m$ -dimensional feature vector.

1.8.3 So the Network Only Learns the Amplitudes — Not the Frequencies?

Exactly. The role of each component is distinct:

- **B fixes the frequencies available** to the network — analogous to the integer harmonics $1, 2, 3, \dots$ in a classical Fourier series, but sampled randomly to provide isotropic coverage of the frequency domain in all coordinate directions simultaneously. **B** is *not* a trainable parameter; it is frozen after initialisation.
- **The MLP weights that follow γ** are the trainable parameters θ . They learn the amplitudes of those basis functions, playing the role of Fourier coefficients. The analogy to a classical Fourier series is direct:

$$f(x) \approx a_0 + \sum_{k=1}^m [a_k \sin(2\pi B_k x) + b_k \cos(2\pi B_k x)] \quad (15)$$

Here the frequencies B_k are fixed; only the coefficients a_k, b_k are fitted. The MLP weights play the role of a_k and b_k .

Because the high-frequency basis functions are explicitly present in the input *before* training begins, the network is no longer required to generate high-frequency variation from smooth activations — a far easier optimisation problem. The standard deviation σ is a hyperparameter controlling the maximum frequency represented; a value of $\sigma \approx 1\text{--}10$ in nondimensional coordinates is typical for thermal field problems.

A useful analogy: **B** sets the piano keys available; the MLP weights learn which keys to press and how hard.

2 Backpropagation and Automatic Differentiation

Two distinct but related differentiation processes operate inside a PINN during training. The first is *backpropagation*, which computes how the loss changes with respect to the network weights θ . The second is *automatic differentiation*, which computes how the network output changes with respect to its coordinate inputs (x^*, r^*, t^*) . Both use the same mathematical tool — the chain rule of calculus — but they ask different questions and serve different purposes. Understanding each clearly is essential before the PINN loss function can be assembled.

2.1 The Computation Graph

Every arithmetic operation a neural network performs is recorded as a node in a *computation graph*. Each node stores two things: the value it computed during the forward pass, and a rule for computing the local derivative during the backward pass. Consider the simplest possible case: a single-neuron network with one weight w , one input x , and a bias b , predicting a target y :

$$z = wx + b, \quad \mathcal{L} = (z - y)^2 \quad (16)$$

The computation graph for this expression has four nodes: a *multiply* node ($w \cdot x$), an *add* node ($(\cdot) + b$), a *subtract* node ($z - y$), and a *square* node ($(\cdot)^2$). During the **forward pass**, values flow left to right and are stored at each node. During the **backward pass**, gradients flow right to left: each node multiplies the incoming gradient by its own local derivative, accumulating the full chain-rule product.

2.2 Backpropagation — Differentiating with Respect to Weights

2.2.1 The Chain Rule

Backpropagation is simply the chain rule applied systematically to the computation graph. For any weight $w_{ij}^{(l)}$ in layer l , the total derivative of the loss is decomposed into a product of local derivatives at each node on the path from $w_{ij}^{(l)}$ to $\mathcal{L}$:

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial z_j^{(l)}}}_{\delta_j^{(l)}} \cdot \underbrace{\frac{\partial z_j^{(l)}}{\partial w_{ij}^{(l)}}}_{=a_i^{(l-1)}} = \delta_j^{(l)} \cdot a_i^{(l-1)} \quad (17)$$

The quantity $\delta_j^{(l)}$ is called the *error signal* at neuron j in layer l . It is computed recursively from the layer above:

$$\delta_j^{(l)} = \sigma'(z_j^{(l)}) \sum_k w_{jk}^{(l+1)} \delta_k^{(l+1)} \quad (18)$$

This recursion is what makes backpropagation computationally efficient: each $\delta_j^{(l)}$ is computed once and reused for every weight in layer l . The total cost is $\mathcal{O}(|\theta|)$ — the same order as one forward pass.

2.2.2 Worked Example: Two Weights, One Step

Consider a two-weight network with inputs $x_1 = 1$, $x_2 = 2$, weights $w_1 = w_2 = 0.5$, bias $b = 0$, and target $y = 4$:

Forward pass:

$$z = w_1x_1 + w_2x_2 + b = 0.5 \times 1 + 0.5 \times 2 + 0 = 1.5 \quad (19)$$

$$\mathcal{L} = (z - y)^2 = (1.5 - 4)^2 = 6.25 \quad (20)$$

Backward pass (applying Eq. 17):

$$\frac{\partial \mathcal{L}}{\partial z} = 2(z - y) = 2(1.5 - 4) = -5.0 \quad (21)$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial w_1} = -5.0 \times x_1 = -5.0 \times 1 = -5.0 \quad (22)$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial w_2} = -5.0 \times x_2 = -5.0 \times 2 = -10.0 \quad (23)$$

Weight update with learning rate $\eta = 0.1$:

$$w_1 \leftarrow 0.5 - 0.1 \times (-5.0) = 1.0 \quad (24)$$

$$w_2 \leftarrow 0.5 - 0.1 \times (-10.0) = 1.5 \quad (25)$$

Verification — new forward pass after one update:

$$z' = 1.0 \times 1 + 1.5 \times 2 = 4.0, \quad \mathcal{L}' = (4.0 - 4)^2 = 0 \quad \checkmark \quad (26)$$

Two observations from this example are important for understanding training behaviour. First, w_2 received a gradient twice as large as w_1 because its input $x_2 = 2$ was twice as large as $x_1 = 1$. Neurons connected to larger inputs receive

larger gradient signals and therefore learn faster — the input magnitude directly sets the learning speed for each connection. Second, both gradients are negative because the network predicted too low ($z = 1.5 < y = 4$), so both weights must increase. The sign of the gradient always points in the direction that increases the loss; gradient *descent* moves opposite to this direction.

2.3 Complex Worked Examples

The following three examples progress from a simple two-gate circuit, through a full sigmoid neuron, to a direct numerical comparison of differentiation methods. Examples 2.3.1 and 2.3.2 are drawn from the open-source course notes of Stanford CS231n [?](#), which are widely used as the canonical reference for backpropagation pedagogy.

2.3.1 Example 1 — $f(x, y, z) = (x + y)z$

Source: Karpathy (2016), CS231n Lecture Notes — Backpropagation ?. Values: $x = -2$, $y = 5$, $z = -4$.

This example uses a two-gate circuit: an *add gate* followed by a *multiply gate*. Decomposing the function into intermediate variables makes each local gradient trivial to compute:

$$q = x + y, \quad f = q \cdot z \tag{27}$$

Forward pass:

$$q = -2 + 5 = 3 \tag{28}$$

$$f = 3 \times (-4) = -12 \tag{29}$$

Local derivatives at each gate:

$$\frac{\partial f}{\partial q} = z = -4, \quad \frac{\partial f}{\partial z} = q = 3 \quad (30)$$

$$\frac{\partial q}{\partial x} = 1, \quad \frac{\partial q}{\partial y} = 1 \quad (31)$$

Backward pass (chain rule applied right to left):

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial x} = (-4)(1) = -4 \quad (32)$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial y} = (-4)(1) = -4 \quad (33)$$

$$\frac{\partial f}{\partial z} = q = \mathbf{3} \quad (34)$$

Two properties of gate behaviour are visible here. First, the *add gate* distributes the upstream gradient equally to both its inputs, since $\partial(x+y)/\partial x = \partial(x+y)/\partial y = 1$ regardless of the input values. Second, the *multiply gate* routes the gradient to each input weighted by the *other* input’s value — the so-called “swapped inputs” rule: $\partial(qz)/\partial q = z$ and $\partial(qz)/\partial z = q$.

2.3.2 Example 2 — Sigmoid Neuron

Source: Karpathy (2016), CS231n Lecture Notes ?. Weights $\mathbf{w} = [2, -3, -3]$, inputs $\mathbf{x} = [-1, -2]$.

A sigmoid neuron computes:

$$f(\mathbf{w}, \mathbf{x}) = \sigma(w_0x_0 + w_1x_1 + w_2) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}} \quad (35)$$

Forward pass:

$$\text{dot} = w_0x_0 + w_1x_1 + w_2 = (2)(-1) + (-3)(-2) + (-3) = 1 \quad (36)$$

$$e^{-\text{dot}} = e^{-1} \approx 0.3679 \quad (37)$$

$$1 + e^{-\text{dot}} \approx 1.3679 \quad (38)$$

$$\sigma = \frac{1}{1.3679} \approx \mathbf{0.731} \quad (39)$$

Rather than backpropagating through each of the four primitive gates (negate, exponentiate, add one, reciprocal), the sigmoid function has a well-known compact derivative:

$$\frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z)) \quad (40)$$

Backward pass (using the grouped sigmoid gradient):

$$\frac{\partial \mathcal{L}}{\partial \text{dot}} = \sigma(1 - \sigma) = 0.731 \times 0.269 \approx \mathbf{0.197} \quad (41)$$

$$\frac{\partial \mathcal{L}}{\partial w_0} = 0.197 \times x_0 = 0.197 \times (-1) = -\mathbf{0.197} \quad (42)$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = 0.197 \times x_1 = 0.197 \times (-2) = -\mathbf{0.394} \quad (43)$$

$$\frac{\partial \mathcal{L}}{\partial w_2} = 0.197 \times 1 = +\mathbf{0.197} \quad (44)$$

Note that w_1 receives exactly twice the gradient magnitude of w_0 because its input $x_1 = -2$ has twice the magnitude of $x_0 = -1$. This confirms the general principle established in the simpler example: input magnitude directly scales the gradient signal that a weight receives.

The compact grouped derivative eq. (40) is an instance of a broader strategy: any sub-expression whose derivative has a simpler closed form than the chain of its primitive gates can be treated as a single gate. Modern frameworks implement this for all standard activation functions.

2.3.3 Example 3 — Autodiff Exactness: $f(x) = \sin(x) e^x$

The exact derivative of $\sin(x)e^x$ is a standard product-rule exercise. The numerical comparison of autodiff against finite differences is adapted from Baydin et al. [?], the survey reference for automatic differentiation in machine learning.

Consider $f(x) = \sin(x) e^x$ evaluated at $x = 1$. Three methods are compared.

Exact derivative (product rule):

$$f'(x) = \cos(x) e^x + \sin(x) e^x = e^x (\sin x + \cos x) \quad (45)$$

$$f'(1) = e^1 (\sin 1 + \cos 1) = 2.71828 \times (0.84147 + 0.54030) = \mathbf{3.75623} \dots \quad (46)$$

Method 1 — Finite differences with $h = 10^{-4}$:

$$\hat{f}'(1) \approx \frac{f(1 + 10^{-4}) - f(1)}{10^{-4}} = \frac{3.75627 \dots - 3.75559 \dots}{10^{-4}} \approx \mathbf{3.75619} \dots \quad (47)$$

$$\text{Error} = |3.75623 - 3.75619| \approx 4.5 \times 10^{-5} \quad (48)$$

Method 2 — Autodiff (computation graph):

The function decomposes into two primitive nodes: $u = \sin(x)$ with $\partial u / \partial x = \cos(x)$, and $v = e^x$ with $\partial v / \partial x = e^x$. Their product $f = uv$ has derivative:

$$\frac{\partial f}{\partial x} = v \cdot \frac{\partial u}{\partial x} + u \cdot \frac{\partial v}{\partial x} = e^x \cos x + \sin x \cdot e^x = e^x (\sin x + \cos x) = \mathbf{3.75623} \dots \quad (49)$$

Error = 0 (to floating-point precision).

Table 1: Comparison of differentiation methods for $f'(1)$ where $f(x) = \sin(x) e^x$.
Exact value: 3.756234...

Method	Result	Error	Cost for $\partial^2 f / \partial x^2$
Finite differences ($h = 10^{-4}$)	3.75619...	$\sim 4.5 \times 10^{-5}$	4 network evaluations, error $\mathcal{O}(h)$
Symbolic differentiation	3.75623...	0	Expression size grows exponentially
Automatic differentiation	3.75623...	0	1 graph traversal, $\mathcal{O}(\text{forward pass})$

For the PINN in this thesis, the PDE residual requires $\partial^2 T^* / \partial x^{*2}$ evaluated at every collocation point. At $N_c = 10,000$ collocation points, finite differences would require $4N_c = 40,000$ network forward passes per training iteration, each introducing $\mathcal{O}(h)$ truncation error that accumulates across the PDE loss. Autodiff computes all N_c second derivatives in a single backward graph traversal at machine precision. This is not a minor optimisation: it is the difference between a computationally feasible and an infeasible training procedure.

2.3.4 Summary: What the Examples Demonstrate

- **Example 1** establishes the two primitive gradient rules that every gate obeys: add gates distribute gradients equally; multiply gates swap their inputs as gradient weights. Every layer in the PINN is a composition of these operations.
- **Example 2** shows that complex activation functions can be treated as single gates once their compact derivative is known. The $\tanh$ derivative $1 - \tanh^2(z)$ plays exactly the same role as the sigmoid derivative $\sigma(1 - \sigma)$: it is the local gradient that the backward pass multiplies by the upstream gradient at every

hidden neuron.

- **Example 3** establishes that autodiff produces exact gradients at the cost of one forward pass, while finite differences produce approximations requiring multiple forward passes. For a PINN with second-order PDE residuals evaluated at thousands of collocation points, this difference determines feasibility.

2.4 Automatic Differentiation — Differentiating with Respect to Inputs

2.4.1 Three Ways to Differentiate

Three methods exist for computing derivatives of a function implemented in code. Only one is suitable for PINNs.

Numerical differentiation (finite differences):

$$\frac{\partial f}{\partial x} \approx \frac{f(x+h) - f(x)}{h} \quad (50)$$

This introduces truncation error $\mathcal{O}(h)$ and numerical cancellation when h is too small. Computing $\partial^2 f / \partial x^2$ requires four network evaluations. For a PINN with millions of collocation points and second-order PDEs, this is both computationally prohibitive and numerically unreliable.

Symbolic differentiation manipulates expressions analytically to produce an exact formula. Exact in principle, but the expression size grows exponentially with the number of operations (*expression swell*), making it impractical for deep networks with millions of operations.

Automatic differentiation (autodiff) applies the chain rule to each elementary operation in the computation graph, accumulating exact numerical derivative values at machine precision — without symbolic manipulation and without truncation error. It is the method used by PyTorch and DeepXDE.

2.4.2 What Makes Autodiff Different from Backpropagation

Backpropagation and autodiff use the same underlying mechanism — the chain rule on a computation graph — but ask different questions:

- **Backpropagation** asks: how does the scalar loss $\mathcal{L}$ change with respect to each weight $w_{ij}^{(l)}$? Answer: $\partial\mathcal{L}/\partial\theta$. This drives learning.
- **Autodiff for PDE residuals** asks: how does the network output $T^*(x^*, r^*, t^*)$ change with respect to each input coordinate? Answer: $\partial T^*/\partial x^*$, $\partial^2 T^*/\partial x^{*2}$, etc. This produces the PDE residual terms.

Both are *reverse-mode* applications of the chain rule. In PyTorch, both are handled by `torch.autograd`. The distinction is in what the differentiation variable is: θ for backpropagation, (x^*, r^*, t^*) for autodiff.

2.4.3 Computing PDE Derivatives in PyTorch

For the axisymmetric heat equation, the PDE residual requires $\partial T^*/\partial t^*$, $\partial^2 T^*/\partial x^{*2}$, and the radial term $\partial^2 T^*/\partial r^{*2} + (1/r^*)\partial T^*/\partial r^*$. In PyTorch these are computed as follows:

```
# Mark coordinates as requiring gradients
x = x.requires_grad_(True)
r = r.requires_grad_(True)
t = t.requires_grad_(True)

# Forward pass
T_pred = network(x, r, t)          # shape: [N, 1]

# First derivatives (create_graph=True keeps the graph for 2nd order)
dT_dx = torch.autograd.grad(
    T_pred, x,
    grad_outputs=torch.ones_like(T_pred),
    create_graph=True)[0]
```

```

dT_dr = torch.autograd.grad(
    T_pred, r,
    grad_outputs=torch.ones_like(T_pred),
    create_graph=True)[0]

dT_dt = torch.autograd.grad(
    T_pred, t,
    grad_outputs=torch.ones_like(T_pred),
    create_graph=True)[0]

# Second derivatives (differentiate dT_dx again w.r.t. x)
d2T_dx2 = torch.autograd.grad(
    dT_dx, x,
    grad_outputs=torch.ones_like(dT_dx),
    create_graph=True)[0]

d2T_dr2 = torch.autograd.grad(
    dT_dr, r,
    grad_outputs=torch.ones_like(dT_dr),
    create_graph=True)[0]

# Assemble the PDE residual (nondimensionalised form)
residual = (dT_dt
            - d2T_dx2
            - A**2 * (d2T_dr2 + dT_dr / r)
            - Q_star)

```

The argument `create_graph=True` is essential. It instructs PyTorch to retain the computation graph of the first derivative so that:

1. The second derivative can be computed (by differentiating `dT_dx` again with respect to `x`), and
2. Gradients can subsequently flow *through* the PDE residual and back to the

weights θ during backpropagation. Without this flag, the second derivative would be computed correctly as a value, but the loss would be disconnected from θ and no learning would occur.

2.4.4 Why Autodiff Is Exact

Consider computing $\partial(\sin x)/\partial x$ numerically at $x = \pi/4$:

$$\text{Exact: } \cos(\pi/4) = 0.70710678\dots \quad (51)$$

$$\text{Finite diff., } h = 10^{-4} : \frac{\sin(\pi/4 + 10^{-4}) - \sin(\pi/4)}{10^{-4}} = 0.70710228\dots \quad (\text{error} \approx 4 \times 10^{-6}) \quad (52)$$

$$\text{Autodiff: } 0.70710678\dots \quad (\text{error} = 0, \text{ machine precision only}) \quad (53)$$

Autodiff achieves this because it does not use h . It evaluates the analytical derivative rule for each elementary operation (the derivative of $\sin$ is $\cos$, the derivative of multiplication by a constant is that constant, and so on) and multiplies these local derivatives together via the chain rule. The result is exact to floating-point precision at every point in the domain. For the PINN PDE residual, this means the physics constraint is imposed exactly — not as a discretisation approximation.

2.5 How Backpropagation and Autodiff Work Together in PINN Training

During each PINN training iteration, both processes operate sequentially:

1. **Forward pass:** the network evaluates $T^*(x^*, r^*, t^*; \theta)$ at all collocation, boundary, and sensor points. All intermediate values are stored in the computation graph.
2. **Autodiff:** `torch.autograd.grad` differentiates T^* with respect to the input coordinates, producing $\partial T^*/\partial t^*$, $\partial^2 T^*/\partial x^{*2}$, etc. These are combined to form the PDE residuals res_1 , the boundary residuals res_2 , and the sensor residuals res_3 .

3. **Loss assembly:** the three residuals are squared, averaged, and combined with adaptive weights $\lambda_1, \lambda_2, \lambda_3$:

$$\mathcal{L}(\theta) = \lambda_1 \|\text{res}_1\|^2 + \lambda_2 \|\text{res}_2\|^2 + \lambda_3 \|\text{res}_3\|^2 \quad (54)$$

4. **Backpropagation:** `loss.backward()` differentiates $\mathcal{L}$ with respect to all weights θ , computing $\nabla_{\theta} \mathcal{L}$ and storing it in `param.grad` for each parameter.
5. **Optimiser step:** Adam (or L-BFGS) updates θ using the accumulated gradients.

Steps 2 and 4 both invoke PyTorch’s autograd engine, but on different variables. This is the complete computational cycle of one PINN training iteration.

3 Convolutional Neural Networks

3.1 What Is a CNN?

A **Convolutional Neural Network** (CNN) replaces the fully connected layers of an MLP with *convolutional filters* — small matrices called *kernels* that slide across the input and detect local patterns. For a one-dimensional signal, a kernel of size k computes the discrete convolution:

$$(f * g)[n] = \sum_j f[j] \cdot g[n - j] \quad (55)$$

This introduces two key structural properties:

- **Weight sharing:** the same kernel is applied at every position in the input. A filter that detects a vertical edge detects it anywhere in the image, using the same weights every time. This reduces the number of trainable parameters dramatically compared to a dense layer.
- **Local connectivity:** each output neuron depends only on a small neighbourhood of the input (the *receptive field*), not on all inputs simultaneously.

The standard CNN architecture stacks convolutional blocks followed by pooling

and dense layers:

$$\text{Input} \rightarrow \left[\text{Conv} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{Pooling} \right]^{\times N} \rightarrow \text{Flatten} \rightarrow \text{Dense} \rightarrow \text{Output}$$

Pooling (max or average) downsamples the feature maps, reducing spatial resolution while increasing the effective receptive field. Early layers detect simple local features (edges, corners); deeper layers combine them into complex patterns.

3.2 Why Does This Thesis Use an MLP and Not a CNN?

This is a conceptually important distinction. CNNs are designed for data defined on a *fixed, discrete grid* — pixel arrays, time series sampled at regular intervals. Their inductive biases (local connectivity, weight sharing, translation equivariance) are only meaningful when the input has spatial structure that repeats or shifts across a grid.

The PINN in this work operates on *continuous coordinates* (x^*, r^*, t^*) , sampled as an unstructured set of collocation points. There is no grid, no neighbourhood, no translational symmetry to exploit. Each forward pass takes a single coordinate triple and returns a scalar temperature — fundamentally a *point evaluation* problem, not a grid-processing problem.

Furthermore, computing the PDE residual requires differentiating the network output with respect to its coordinate inputs: $\partial f / \partial x^*$, $\partial^2 f / \partial x^{*2}$, $\partial f / \partial r^*$, and so on. In a CNN the input is a discrete array and differentiation in the continuous sense is not defined. In the MLP the coordinates (x^*, r^*, t^*) are continuous real-valued inputs, and their derivatives are computed exactly by automatic differentiation (Section 4).

Table 2 summarises the structural differences.

Table 2: Comparison of CNN and MLP for the PINN application.

Property	CNN	MLP (this work)
Input structure	Fixed grid (pixels, voxels)	Unstructured point set
Spatial assumption	Local patterns, translation equivariance	None — global function approximation
Differentiation w.r.t. input	Not meaningful (discrete input)	Core operation (PDE residual)
Weight sharing	Yes — efficient for grid data	No — each weight is distinct
Thesis relevance	Background only	context Core architecture

CNNs do appear in adjacent PDE-solving literature — convolutional surrogate models for computational fluid dynamics, for instance — but these approaches discretise the domain onto a grid first, losing the mesh-free advantage that motivates PINNs.

3.3 Summary: MLP vs CNN for Continuous Coordinate Input

The choice of MLP over CNN is not a matter of performance preference — it is structurally required by the problem. The PINN must:

1. Accept a continuous coordinate triple (x^*, r^*, t^*) as input;
2. Return a scalar field value $T^*(x^*, r^*, t^*)$;
3. Support exact second-order differentiation of its output with respect to its inputs via automatic differentiation.

Only a fully connected MLP with smooth activations satisfies all three requirements. A CNN satisfies none of them in the continuous-coordinate setting.

The next section introduces *automatic differentiation* — the mechanism that makes requirement 3 possible and that forms the bridge between the MLP architecture described above and the PINN loss function.

4 Automatic Differentiation

Automatic differentiation (autodiff) is the computational mechanism that makes PINNs possible. It is distinct from both numerical finite differences and symbolic differentiation, and understanding it is essential before the PINN loss function can be assembled.

4.1 Three Ways to Differentiate — Why Only One Works for PINNs

Numerical differentiation (finite differences):

$$\frac{\partial f}{\partial x} \approx \frac{f(x+h) - f(x)}{h} \quad (56)$$

This introduces truncation error of order $\mathcal{O}(h)$ and numerical cancellation when h is small. Computing $\partial^2 f / \partial x^2$ requires four network evaluations. For a PINN with millions of collocation points and second-order PDEs this is computationally prohibitive and numerically unreliable.

Symbolic differentiation manipulates expressions analytically to produce an exact formula. Exact in principle, but the expression size grows exponentially with the number of operations (*expression swell*), making it impractical for deep networks.

Automatic differentiation applies the chain rule to each elementary operation in the computation graph, accumulating exact numerical derivative values at machine precision — without symbolic manipulation and without truncation error.

4.2 How Autodiff Computes PDE Residuals in PyTorch

For the PINN loss, the required derivatives are $\partial f / \partial x^*$ and $\partial^2 f / \partial x^{*2}$, where f is the network output and x^* is an input coordinate. In PyTorch these are computed as:

```
# First derivative dT/dx
dT_dx = torch.autograd.grad(
    T, x, grad_outputs=torch.ones_like(T),
    create_graph=True)[0]

# Second derivative d^2T/dx^2 (differentiate again)
d2T_dx2 = torch.autograd.grad(
    dT_dx, x, grad_outputs=torch.ones_like(dT_dx),
    create_graph=True)[0]
```

The argument `create_graph=True` is essential: it keeps the computation graph of the first derivative alive so that the second derivative can be computed and, crucially, so that gradients can flow *through* the PDE residual back to the weights θ during backpropagation. This is what connects training (Section 13) to the physics: the PDE residual is a differentiable function of θ , so the entire PINN loss is minimisable by standard gradient descent.

The key point is that autodiff does not approximate — it computes $\partial f / \partial x^*$ to machine precision. The PDE residual is therefore an exact constraint on the network, not a discretised approximation.